Ecodrive - Vizsgaremek Dokumentáció

Tartalomjegyzék

1. Modul: Bevezetés és a projekt előkészítése 4
1.1. A projekt célja és alapötlete 4
1.2. A csapat és a szerepkörök kiosztása 4
2. Modul: Rendszerarchitektúra és Fejlesztői Környezet 5
2.1. Technológiák 5
2.2. A fejlesztői környezet evolúciója: XAMPP-ról a Dockerre 5
3. Modul: Az Autentikációs Rendszer és Biztonság 7
3.1. Az Authenticator Végpont és a Bemenet-ellenőrzés 7
3.2. Jelszókezelés és az Argon2i Algoritmus 7
3.3. Adatbázis-vezérelt Munkamenetek 8
4. Modul: Járműkezelés és Útvonalnaplózás 9
4.1. Adatmodellek: A Vehicle és a Route entitások 9
4.2. Szigorú Bemenet-ellenőrzés és Reguláris Kifejezések 9
4.3. Biztonság: IDOR Védelem és HTTP Státuszkódok 10
4.4. A CO_2 Számítás Logikája és a Kaszkádolt Törlés 10
5. Modul: Infrastruktúra, Adatbázis-tervezés és Konfiguráció 11
5.1. Konténerizált Fejlesztői Környezet (Docker) 11
5.2. Adatbázis Architektúra és Relációk 12
5.3. Konfiguráció és Környezeti Változók 12
6. Modul: Minőségbiztosítás és Szoftvertesztelés 13
6.1. Tesztelési szintek és a Tesztpiramis alkalmazása 13
6.2. Teszttervezési Technikák a gyakorlatban 13
7. Modul: Az Adminisztrációs Felület és Jogosultságkezelés 15
7.1. Adatbázis szintű szerepkörök 15
7.2. Backend routing és Middleware-szerű védelem 15
7.3. UI Adaptáció és Rendszerkezelés 15
8. Modul: Statisztikai Elemzés és Extrapoláció (A Legkisebb Négyzetek Módszere) 16
8.1. Adatok betáplálása és Aggregáció (A feed metódus) 16
8.2. A Lineáris Regresszió Matematikája (A finalise metódus) 16
8.3. Predikció Integrálszámítással (Az accumulate metódus) 17

9. Modul: Összegzés és a Csapatmunka Értékelése 18
FORRÁSJEGYZÉK / FÁJLLISTA 19
1. Infrastruktúra és Konfiguráció 19
3. Adatmodellek (Models) 19
4. Végpontok és Üzleti Logika (Endpoints) 19
5. Segédosztályok és Algoritmusok (Helpers) 20

1. Modul: Bevezetés és a projekt előkészítése

1.1. A projekt célja és alapötlete

A modern társadalom egyik legnagyobb kihívása a környezettudatosság és a fenntartható közlekedés kérdése. Projektünk, az "Ecodrive" webes alkalmazás erre a problémára kínál egy átfogó, digitális megoldást. A szoftver célja, hogy az autósokat segítse járműveik üzemanyag-fogyasztásának, a megtett útjaiknak, valamint a CO□-kibocsátásuknak a nyomon követésében. A rendszer nem csupán egy egyszerű naplózó felület; egy olyan komplex ökoszisztéma, amely a bevitt adatokból áttekinthető statisztikákat generál, lineáris extrapolációval éves becsléseket készít, és a felhasználó kibocsátását összeveti az európai átlaggal. Ezzel a funkcióval a felhasználók közvetlen visszajelzést kapnak arról, hogy mennyire közlekednek környezetbarát módon.

A szoftver funkciói közé tartozik a biztonságos regisztráció és bejelentkezés, a felhasználókhoz rendelt járművek kezelése, az útvonaltervezés, valamint a ténylegesen megtett utak tárolása és elemzése.

1.2. A csapat és a szerepkörök kiosztása

A fejlesztés legelején, a tervezési fázisban a csapat összeült, hogy közösen határozza meg a mérföldköveket, a felhasználandó technológiákat és a feladatköröket. A hatékony munka érdekében a feladatokat élesen elválasztottuk, ugyanakkor a modulok összekötése folyamatos kommunikációt igényelt. A projekt tagjai és főbb feladataik a következők voltak:

- **Deák Benedek (Frontend és UI/UX):** a teljes felhasználói felület (HTML, CSS) és a PHP-alapú nézetréteg kialakítása. Ő felelt a bejelentkezési, regisztrációs, profilkezelő, valamint az autós és útvonalnaplózó űrlapok reszponzív és felhasználóbarát megjelenéséért, továbbá az admin felület vizuális adaptációjáért.
- **Szabó Zsigmond Károly (Backend logika és segéd):** a szerveroldali logika (PHP) és a matematikai/statisztikai számítások megírása. Az ő feladata volt a login adatok, a járművek és az utazási naplók adatainak biztonságos szerveroldali kezelése, valamint az ehhez tartozó jogosultság-ellenőrzések implementálása.
- **Széplaki Dániel (Adatbázis, DevOps és Tesztelés):** az adatbázis (MySQL) struktúrájának megtervezése, valamint a teljes tesztelési dokumentáció elkészítése. Ő vezényelte le a fejlesztői környezet virtualizációját is.

A munkafolyamat egy részletes projektprezentáció elkészítésével indult, amelyben vizuálisan is prezentáltuk a koncepciót, a drótvázakat és az adatbázis terveit. Ez a dokumentum szolgált később a tényleges megvalósítás és a jelen dokumentáció alapjául is. A legutolsó lépés az adminisztrációs felület elkészítése volt, amelyet konzulensi egyeztetés előzött meg, hogy az minden vizsgakövetelménynek megfeleljen.

2. Modul: Rendszerarchitektúra és Fejlesztői Környezet

2.1. Technológiák

A projekt technológiai hátterét úgy választottuk meg, hogy az biztosítsa a stabilitást, az adatok biztonságos kezelését, és a könnyű bővíthetőséget.

- **Frontend:** a megjelenítésért hagyományos HTML5 és CSS3 technológiák feleltek.
- **Backend:** az program logikát PHP nyelven írtuk. A PHP felelt a kérések (GET, POST) fogadásáért, a formok validációjáért, a session-ök kezeléséért, komplexebb számítások elvégzéséért, valamint a kommunikációért az adatbázissal.
- **Adatbázis:** relációs adatbázis-kezelőként a MySQL-t használtuk.

2.2. A fejlesztői környezet evolúciója: XAMPP-ról a Dockerre

A projekt kezdeti fázisában a fejlesztés egy hagyományos XAMPP (Apache + MariaDB/MySQL + PHP) környezetben indult el. Azonban a csapatmunka során hamar komoly technikai akadályokba ütköztünk. Benedek laptopján a XAMPP specifikus verziókompatibilitási és konfigurációs hibákat produkált, ami miatt a PHP kódok és az adatbázis-kapcsolatok nem futottak megfelelően, jelentősen hátráltatva a frontend és a backend integrációját.

A lokális gépek közötti különbségek (különböző operációs rendszerek, eltérő PHP és MySQL verziók, környezeti változók) megszüntetése érdekében Dániel javaslatára a csapat meghozta a technológiai döntést: a teljes projektet áthelyezzük egy virtuális konténerizált környezetbe, a **Docker** platformra. Ez a váltás nem csak a kompatibilitási problémákat oldotta meg Benedek gépén, de a projekt szerkezetét is sokkal letisztultabbá és elkülönítettebbé tette.

A Docker környezet felépítése egy `compose.dev.yml` fájlban lett definiálva. A rendszer három fő konténerből állt össze:

- **app-server:** ez a konténer tartalmazta a PHP környezetet és magát az alkalmazást. (A konfiguráció során Dániel javított egy tipikus fejlesztési hibát is, amikor az *„app-server-hez nincs image vagy build context”* hibaüzenet jelentkezett; ennek megoldásával biztosította, hogy minden tagnál ugyanaz a lemezkép épüljön fel.)
- **MySQL konténer:** egy teljesen izolált adatbázis-szerver, amely független volt a hoszt gép lokális telepítéseitől.

- **phpMyAdmin konténer:** egy grafikus felület az adatbázis kényelmes adminisztrációjához.

A környezet indítása a `docker compose -f compose.dev.yml up –build` paranccsal történt. Ez az egységesítés garantálta, hogy a weboldal mindig a `http://localhost:8888` címen, a phpMyAdmin a `http://localhost:8080` porton, míg maga a MySQL adatbázis a standard 3306-os porton legyen elérhető minden csapattag számára, megszüntetve ezzel a "*nálam működik, nálad miért nem?"* típusú fejlesztői anomáliákat.

3. Modul: Az Autentikációs Rendszer és Biztonság

A felhasználók adatainak védelme minden modern webes alkalmazás alapkövetelménye. Az Ecodrive projektben a biztonsági architektúra felépítése Zsigmond (backend végpontok és titkosítás), Dani (adatbázis-alapú munkamenetek) és Beni (frontend validációk) szoros együttműködésének eredménye. A cél az volt, hogy a rendszer ne csak funkcionálisan működjön, hanem ellenálljon az olyan gyakori sebezhetőségeknek is, mint az SQL injekció (SQLi), a Cross-Site Scripting (XSS) vagy a Session Hijacking.

3.1. Az Authenticator Végpont és a Bemenet-ellenőrzés

A bejelentkezési és regisztrációs folyamatokat Zsigmond az `./app/endpoints/Authentication.php` fájlban definiált `Authenticator` osztályban valósította meg, amely implementálja a rendszer közös `Endpoint` interfészét.

A `processRegistration()` metódus szigorú, többlépcsős validációt végez a bemeneti adatokon. Mielőtt az adatok az adatbázis közelébe kerülnének, a rendszer az `EcoDrive\Helpers\UserDataValidator` segítségével ellenőrzi a felhasználónév formai követelményeit, az e-mail cím struktúráját (a beépített `filter_var()` segítségével), valamint a jelszavak komplexitását és egyezőségét. A bejelentkezésnél a rendszer képes azonosítani a felhasználót akár a felhasználóneve, akár az e-mail címe alapján. A PHP backend intelligensen eldönti, hogy milyen típusú adatot kapott, és ennek megfelelően dinamikusan választja ki az adatbázis-lekérdezés típusát. Ha az azonosítás sikertelen, a rendszer szándékosan egy általános hibaüzenetet ad (*"A felhasználónév vagy jelszó helytelen"*), megakadályozva ezzel a felhasználónév-enumerációs támadásokat.

3.2. Jelszókezelés és az Argon2i Algoritmus

A felhasználói adatok (entitások) kezelését az `./app/models/User.php` modell végzi. A jelszavak tárolása kritikus biztonsági kérdés. Zsigmond a fejlesztés során nem a régebbi (és mára elavultnak számító) MD5 vagy SHA-256 algoritmusokat használta, és még csak nem is a PHP alapértelmezett Bcrypt-jét.

A kódban explicit módon az Argon2i hashfüggvény került használatra a password_hash() függvényben. Az Argon2i egy memóriát (memory-hard) és processzort (CPU-hard) intenzíven használó algoritmus, amely rendkívül erős védelmet nyújt mind a brute-force, mind a szótáralapú, mind pedig a dedikált hardveres (ASIC/GPU) támadások ellen. A bejelentkezéskori jelszó-ellenőrzést a `password_verify()` függvény végzi, ami biztonságos, időzítés-független (timing-attack resistant) összehasonlítást garantál. További védelmi réteg, hogy a Model interfész `modelEscaped()` metódussal is rendelkezik, ami a nézetrétegbe történő kiíratás előtt védekezik az XSS (Cross-Site Scripting) injekciók ellen.

3.3. Adatbázis-vezérelt Munkamenetek

A projekt egyik legfejlettebb technikai megoldása a munkamenetek (session) kezelése, amelyet Zsigmond és Dani az ./app/models/Session.php fájlban tervezett meg. Ahelyett, hogy a hagyományos, szerver fájlrendszerére támaszkodó PHP sessionökre (PHPSESSID) hagyatkoztunk volna, a csapat egy egyedi, perzisztens, adatbázis-alapú rendszert hoztunk létre.

Amikor a jelszó-ellenőrzés sikeres, a createSessionForUser() metódus lép működésbe. A rendszer a beépített random_bytes() kriptográfiai függvénnyel generál egy 32 bájtos, teljesen véletlenszerű azonosítót, amelyet base64 kódolással alakít át olvasható stringgé. Ez az azonosító (a token) és annak pontos lejárati ideje (amely a DateTimeImmutable osztállyal számolva pontosan egy hét a generálás pillanatától) bekerül a sessions adatbázistáblába.

Ezt követően az initialiseSessionCookie() metódus elküldi a tokent a felhasználó böngészőjének. Itt egy kiemelkedően fontos biztonsági beállítást alkalmaztunk: a session azonosítót tároló sütin beállítottuk a HttpOnly flaget. Ennek köszönhetően az Ecodrive session sütije kliensoldali JavaScriptből (pl. document.cookie) egyáltalán nem olvasható ki, így egy esetlegesen sikeres XSS támadás esetén is a támadó képtelen ellopni a felhasználó munkamenetét (session hijacking elleni védelem). A kijelentkezéskor a rendszer nemcsak a sütit törli, hanem aktívan megsemmisíti az adatbázisban lévő token-rekordot is, azonnal érvénytelenítve az összes lehetséges hozzáférést az adott munkamenethez.

4. Modul: Járműkezelés és Útvonalnaplózás

Az Ecodrive alkalmazás legfőbb program logikája a felhasználók járműveinek és a hozzájuk tartozó utazásoknak a kezelése. Ennek a modulnak a megvalósítása tökéletesen példázza a csapaton belüli munkamegosztást: az adatbázis-modelleket Dani optimalizálta, a komplex szerveroldali üzleti logikát és a védelmeket Zsigmond írta, míg az adatok vizuális és interaktív tálalásáért Beni felelt a frontend oldalon.

4.1. Adatmodellek: A Vehicle és a Route entitások

Az adatbázisban tárolt információk objektumorientált leképezésére az ./app/models/Vehicle.php és a Route.php fájlokat hoztuk létre. A járművek esetén a rendszer rögzíti a márkát, modellt, rendszámot, az évjáratot és a fogyasztási adatokat. Kiemelendő egy fontos program szabály, amelyet Zsigmond a modellben konstansként definiált: DEFAULT_EMISSION_RATE = 108.2. Ez az érték az Európai Unió átlagos gépjármű-kibocsátási normája (g/km), amelyet a rendszer alapértelmezettként használ, ha a felhasználó nem ismeri a saját autójának pontos emissziós adatát.

A táblák közötti szoros relációs kapcsolatot (1:N) a modellek szintjén is lekezeltük. A Route osztály példányosításakor a rendszer automatikusan betölti a hozzá tartozó Vehicle objektumot is. Az adatbázis-műveletek (CRUD) során Zsigmond minden lekérdezésnél (pl. createVehicleQuery() , existsQuery()) **prepared statement**-eket használt a mysqli_stmt_prepare() és mysqli_stmt_bind_param() függvényekkel. Ez a megközelítés 100%-os védelmet nyújt a klasszikus SQL Injection (SQLi) támadásokkal szemben, mivel a bemeneti adatok soha nem lesznek futtatható SQL parancsként értelmezve.

4.2. Szigorú Bemenet-ellenőrzés és Reguláris Kifejezések

A járművek létrehozását és frissítését az ./app/endpoints/Vehicles.php végpont kezeli. Zsigmond itt egy rendkívül részletes, többlépcsős validációs rendszert épített ki (validateFields), amely biztosítja, hogy az adatbázisba csak tiszta és logikailag helyes adatok kerüljenek:

- **Évjárat validáció (validateYear()):** A rendszer nemcsak a típusmegfelelőséget ellenőrzi (FILTER_VALIDATE_INT), hanem logikai határérték-vizsgálatot is tart: az évjárat nem lehet kisebb, mint 1900, és nem lehet nagyobb, mint a szerver aktuális éve.
- **Rendszám ellenőrzés (validatePlate()):** A magyar rendszámok validálására komplex regexet alkalmaztunk. A kód helyesen ismeri fel mind a régebbi (3 betű - 3 szám, az új típusú (2 betű - 2 betű - 3 szám) formátumokat, továbbá a magyar jogszabályokban meghatározott egyedi rendszámok formátumait is.

- **Típuskonverziók:** Mivel a magyar felhasználók a tizedestörteket gyakran vesszővel adják meg, a rendszer automatikusan elvégzi a vessző cseréjét pontra, hogy a PHP float típusúként tudja értelmezni a fogyasztási és emissziós adatokat.

Ezekre az ellenőrzésekre a felhasználó a UI-on keresztül azonnali, specifikus hibaüzeneteket kap (pl. *"A rendszám már foglalt"*), amelyeket Beni tervezett meg reszponzív HTML formában.

4.3. Biztonság: IDOR Védelem és HTTP Státuszkódok

A webes rendszerek egyik leggyakoribb sebezhetősége az IDOR (Insecure Direct Object Reference), amikor egy felhasználó egy egyszerű URL-paraméter átírásával hozzáférhetne mások adataihoz.

A `Vehicles` osztály `update()` és `delete()` metódusaiban Zsigmond implementált egy kritikus védelmi vonalat: a rendszer minden adatmódosítás kísérletnél ellenőrzi, hogy a módosítani kívánt jármű (amit a kliens pl. a `vehicleId` paraméterben küld) ténylegesen az éppen bejelentkezett felhasználóhoz (`Session::currentUser()`) tartozik-e. Ha nem, az alkalmazás megtagadja a kérést, és megfelelő HTTP státuszkóddal (401 Unauthorized) válaszol. Az API tervezése a RESTful elveket követi: sikeres műveletnél 200 OK, logikai hibánál (pl. hiányzó jármű) 422 Unprocessable Entity, nem létező azonosítónál pedig 404 Not Found fejléceket küld a backend.

4.4. A CO_2 Számítás Logikája és a Kaszkádolt Törlés

A felhasználók utazásainak felvitelekor (`Route::create()`) az Ecodrive legfontosabb program logikája fut le. Amikor Beni felületén a felhasználó megadja a kiindulási és célállomást (irányítószám, város, utca), valamint a megtett távolságot, a rendszer szerveroldalon kalkulálja ki a kibocsátást (ami a megtett távolság és az adott gépjármű kibocsátási rátájának szorzata). Fontos, hogy ez a számítás a backend-en, a megbízható járműadatokból történik, így a kliens nem tudja manipulálni (például 0 kibocsátást beküldeni) az eredményt.

Az adatok összekapcsoltsága miatt a törlési folyamatokat körültekintően kellett megtervezni. A `Vehicles.php` modellben a `delete()` metódus hívása esetén a rendszer először lefutatja a `Route::deleteRoutes()` függvényt. Ez az alkalmazás-szintű kaszkádolt törlés garantálja az adatbázis hivatkozási integritását: egy autó törlésekor a hozzá tartozó összes naplózott út is törlődik, elkerülve ezzel a "beragadt" vagy árva rekordokat az adatbázisban. Mind a járműveknél, mind az útvonalaknál alkalmazásra került a `modelEscaped()` függvény, amely a HTML kimenet generálása előtt kicseréli a speciális karaktereket, így egy sikeres XSS támadás esélyét nullára csökkenti.

5. Modul: Infrastruktúra, Adatbázis-tervezés és Konfiguráció

A projekt technikai alapjait a stabilitás, a biztonság és a skálázhatóság elvei mentén terveztük meg. Széplaki Dániel feladata volt egy olyan részletes háttérrendszer kialakítása, amely nemcsak a fejlesztés során jelentkező anomáliákat küszöböli ki, de egy éles környezet követelményeinek is meg tudna felelni.

5.1. Konténerizált Fejlesztői Környezet (Docker)

Ahogy a korábbi modulokban említettük, a lokális fejlesztési környezetek (XAMPP) okozta kompatibilitási problémák miatt a csapat a Docker alapú virtualizáció mellett döntött. A rendszer magját a `compose.dev.yml` fájl határozza meg, amely három elkülönített szolgáltatást (service) indít el:

- **app-server (PHP 8.3 + Apache):** Ez a konténer felel a backend futtatásáért. A build: context utasítás révén a saját, egyedi Dockerfileunk alapján épül fel. Fejlesztés közben `a volumes: - .:/var/www/html` beállítás biztosítja a "live reload" élményt: a gazdagépen módosított fájlok azonnal szinkronizálódnak a konténerbe, így nem kell minden kódmódosítás után újraépíteni a lemezképet.
- **database (MySQL 9.5):** A relációs adatbázis-kezelő. Kiemelendő a perzisztencia megoldása: a db-data nevű Docker volume biztosítja, hogy a konténer leállítása vagy törlése esetén se vesszenek el az adatok. Az adatbázis inicializálása teljesen automatizált: a database.sql fájlt a rendszer a `/docker-entrypoint-initdb.d/init.sql` könyvtárba csatolja be, így az első indításkor a táblák automatikusan létrejönnek.
- **phpMyAdmin:** Egy dedikált konténer a 8080-as porton, amely közvetlen, grafikus hozzáférést biztosít az adatbázishoz a tesztelések és az adminisztráció megkönnyítése érdekében.

A **Dockerfile** felépítésénél Dániel különös figyelmet fordított a biztonságra és a teljesítményre. A hivatalos php:8.3-apache alapképre telepítésre kerültek a szükséges adatbázis-illesztők (mysqli, pdo_mysql), valamint engedélyezve lett az Apache rewrite modulja, amely a tiszta, SEO-barát URL-ek és a backend routing (végpont-kezelés) elengedhetetlen feltétele. Szakmai szempontból kiemelkedő biztonsági lépés, hogy az alkalmazás nem "root" felhasználóként fut a konténeren belül. `A COPY --chown=www-data:www-data` utasítás és a USER www-data beállítás garantálja a legkisebb jogosultság elvét.

5.2. Adatbázis Architektúra és Relációk

Az Ecodrive adatbázisa (database.sql) relációs elvekre épül, és hivatkozási integritást alkalmaz. A karakterkódolásnál a utf8mb4_unicode_ci formátumot használtuk, amely nemcsak a speciális magyar karakterek, de akár az emojik hibátlan tárolását is biztosítja.

A rendszer négy fő táblából áll:

- **users:** A felhasználók adatait tárolja. A hagyományos adatok (username, email) mellett tartalmazza a jelszó-visszaállításhoz szükséges tokeneket (reset_token, reset_token_expiry), valamint egy is_admin logikai flaget a jogosultságkezeléshez. A biztonság magas fokát jelzi, hogy a rendszer a beépített adminisztrátor jelszavát a rendkívül erős, iparági standardnak számító Argon2i algoritmussal hashelve tárolja.
- **sessions:** A munkamenetek kezelése nem a PHP alapértelmezett (fájlalapú) módszerével történik, hanem adatbázis-vezérelt. Ez a tábla köti össze a felhasználókat a generált session_id-val, és tárolja a lejárati időt. Ez a megoldás nagyságrendekkel biztonságosabb, és megakadályozza a session-fixation támadásokat.
- **vehicles:** A gépjárművek adatai (márka, modell, rendszám). A rendszám (license_plate) mező UNIQUE megkötést kapott az adatduplikáció elkerülése végett.
- **routes:** Az utazási napló bejegyzései. Rögzíti az indulási és érkezési pontokat, a távolságot és a számított CO□-kibocsátást. Ez a tábla a vehicles táblához kapcsolódik idegen kulcs kapcsolattal.

5.3. Konfiguráció és Környezeti Változók

A modern szoftverfejlesztési alapelveknek megfelelően az alkalmazás konfigurációs adatai nem a forráskódban szerepelnek. Ezt a feladatot a .env fájl látja el, amely tartalmazza a bizalmas adatokat (DB jelszó, host). Ezeket az adatokat az ./app/config.php fájlban definiált Config osztály olvassa be. Ez az objektumorientált konfiguráció-kezelés biztosítja, hogy a backend további moduljai tiszta, átlátható módon férhessenek hozzá a központi beállításokhoz.

6. Modul: Minőségbiztosítás és Szoftvertesztelés

Egy professzionális szoftverfejlesztési folyamat elengedhetetlen része a minőségbiztosítás (QA). A projekt során Dani dolgozta ki a szoftvertesztelési stratégiát, amely biztosította, hogy az Ecodrive alkalmazás ne csak a "happy path" (ideális működés) mentén működjön hibátlanul, hanem a váratlan felhasználói interakciókat és a környezeti anomáliákat is kezelni tudja. A tesztelési keretrendszer a hagyományos tesztpiramis elveire épült, ötvözve az elméleti módszertanokat a gyakorlati megvalósítással.

6.1. Tesztelési szintek és a Tesztpiramis alkalmazása

A projekt minőségbiztosítása több, egymásra épülő szinten valósult meg:

- **Unit tesztelés szemlélete:** Bár a projekt natív PHP-ban készült, az üzleti logika szigorú szétválasztása (MVC architektúra) lehetővé tette a modulok izolált vizsgálatát. Különös figyelmet kapott a Zsigmond által írt `LeastSquaresExtrapolator` és a `UserDataValidator` osztály. Itt a tiszta matematikai és üzleti szabályok érvényesülését vizsgáltuk (pl. nullával való osztás kezelése a regressziós számításnál).
- **Integrációs tesztelés:** Mivel a rendszer több komponensből (PHP backend, MySQL adatbázis, Python preprocesszor) áll, a rendszerhatárok tesztelése kritikus volt. Az integrációs tesztek során derült ki a lokális XAMPP környezetek okozta inkompatibilitás, ami végül a Docker bevezetéséhez vezetett. A konténerizáció után a backend és az adatbázis közötti kapcsolat (PDO/MySQLi) már egy stabil, éleshez közeli környezetben volt tesztelhető.
- **End-to-End (E2E) és Smoke tesztelés:** A teljes felhasználói utat lefedő E2E teszteket hajtottunk végre. Egy tipikus E2E teszteset (Smoke test) magában foglalta a regisztrációt, a bejelentkezést, egy új jármű felvitelét, majd egy útvonal rögzítését és a statisztika generálását. Ha ez a folyamat megszakadt, a release nem történhetett meg.

6.2. Teszttervezési Technikák a gyakorlatban

A bemeneti mezők (Beni UI formjai és Zsigmond szerveroldali validációi) ellenőrzésére klasszikus teszttervezési technikákat alkalmaztunk a redundancia elkerülése és a tesztlefedettség növelése érdekében:

- **Ekvivalenciaosztály-képzés:** A rendszámok formátumának ellenőrzésénél (a `validatePlate()` metódusban) a bemeneteket érvényes (pl. ABC-123, AA-BB-123) és érvénytelen (pl. AB-1234, ABCDE) halmazokra bontottuk, és minden halmazból vettünk mintát a teszteléshez.

- **Határérték ellenőrzés:** Az évjárat ellenőrzésénél az 1900-as és az aktuális év jelentették a határokat. A tesztek során beküldtük az 1899-es (érvénytelen alsó), az 1900-as (érvényes alsó), az aktuális évet (érvényes felső), valamint a jövő évet (érvénytelen felső), biztosítva, hogy a logika minden edge caset lekezel.
- **Biztonsági és Jogosultsági Negatív tesztek:** Explicit módon teszteltük, mi történik, ha egy hitelesített felhasználó egy másik felhasználó járművét próbálja módosítani (IDOR támadás szimulálása). A teszt sikeres volt, mivel a backend ilyen esetben a specifikációnak megfelelően 401 Unauthorized hibát dobott.

7. Modul: Az Adminisztrációs Felület és Jogosultságkezelés

A projekt utolsó, de vizsgakövetelmény szempontjából kiemelten fontos mérföldköve a dedikált adminisztrációs felület kialakítása volt. Ennek megvalósítása igazi csapatmunka volt, amelyet egy konzulensi egyeztetés előzött meg, hogy pontosan definiáljuk az adminisztrátorok szerepkörét és jogosultsági határait az Ecodrive ökoszisztémában.

7.1. Adatbázis szintű szerepkörök

A jogosultságkezelés alapjait Dani fektette le a database.sql fájlban. A users táblát kibővítette egy is_admin logikai mezővel, amelynek alapértelmezett értéke FALSE. Annak érdekében, hogy a rendszer telepítésekor azonnal rendelkezésre álljon egy szuperfelhasználó, egy inicializáló szkripttel létrehozott egy admin felhasználót, amelynek jelszavát – a normál felhasználókhoz hasonlóan – biztonságos Argon2i hash-sel védte.

7.2. Backend routing és Middleware-szerű védelem

Az admin végpontok védelmét Zsigmond implementálta a PHP backendben. A rendszer Endpoint interfésze kötelezővé teszi az isAdminPermissible() statikus metódus megvalósítását minden útvonal esetében. A dedikált Admin.php végponton ez a metódus true értéket ad vissza, viszont a kérések feldolgozása előtt a rendszer ellenőrzi a Session-ben tárolt aktív felhasználó is_admin flagjét. Ha egy normál felhasználó próbál meg hozzáférni az admin felülethez, a rendszer azonnal megtagadja a kérést és átirányítja a főoldalra, megakadályozva a vertikális jogosultság-eszkalációt. Az adminisztrátor egy speciális felhasználó: olyan jogosultságokkal rendelkezik, amelyekkel más felhasználók adatait tudja kezelni, de ő maga nem használhatja az alkalmazást a normál felhasználókhoz hasonló módon (tehát pl. nem naplózhatja saját útvonalait). Éppen ezért az isAdminPermissible() falset ad vissza a legtöbb végponton; ha egy adminisztrátor egy ilyen végpontra látogat, a rendszer visszairányítja az admin felületre.

7.3. UI Adaptáció és Rendszerkezelés

A frontend vizuális kialakításáért Beni felelt. Annak érdekében, hogy a projekt határidőre elkészüljön és a kód újrahasznosíthatósága (DRY elv - Don't Repeat Yourself) megmaradjon, Beni a már meglévő, reszponzív HTML+CSS nézeteket (views) használta fel alapként. Az admin felületen (Dashboard) egyedi táblázatszerű elrendezéseket és extra szűrőket helyezett el, amelyeken keresztül az adminisztrátor átláthatja a regisztrált felhasználók listáját, a rendszerbe rögzített járműveket, és szükség esetén beavatkozhat (például törölhet inaktív fiókokat vagy frissítheti felhasználók adatait). Ez a felület logikusan illeszkedik a projekt egészébe, vizuálisan koherens a felhasználói oldallal.

8. Modul: Statisztikai Elemzés és Extrapoláció (A Legkisebb Négyzetek Módszere)

Az Ecodrive alkalmazás leginnovatívabb és legkomplexebb backend funkciója a statisztikai modul, amely nem csupán rögzíti a múltbéli adatokat, hanem prediktív elemzést is végez. A cél az volt, hogy a rendszer a felhasználó eddigi utazási szokásaiból matematikai úton megbecsülje az éves, összesített CO□-kibocsátást. Ezt a feladatot az ./app/helpers/LeastSquaresExtrapolator.php osztály látja el, amely tisztán natív PHP kódban, külső matematikai könyvtárak mellőzésével valósítja meg a lineáris regressziót.

8.1. Adatok betáplálása és Aggregáció (A feed metódus)

A statisztikai motor első lépése a nyers utazási adatok (amelyeket Dani SQL aggregációs lekérdezései szolgáltatnak) betáplálása a feed() függvénnyel. Az X tengely az eltelt időt (például az év napjait), míg az Y tengely a kibocsátott CO□ mennyiséget reprezentálja. Zsigmond itt aggregációt alkalmazott: ha egy adott X ponton (pl. ugyanazon a napon) már létezik adat, a rendszer nem írja felül azt, hanem hozzáadja az új Y értéket (tehát ha adott napon több útvonal lett naplózva, akkor ezen kibocsátások összege lesz az adott Y érték).

8.2. A Lineáris Regresszió Matematikája (A finalise metódus)

Miután a pontfelhő bekerült a memóriába, a finalise() metódus hívásával lefut a **Legkisebb Négyzetek Módszere**. Ez az algoritmus megkeresi azt az egyenest amely a legkisebb hibával illeszkedik a mért adatpontokra. Ha az egyenes $y = mx + b$, akkor a legjobban illeszkedő egyenes (ebben a kontextusban) az, amelyik a hibák négyzetösszegét minimalizálja:

$$\sum_k (mx_k - y_k + b)^2$$

Ebből kiszámolható a meredekség és a tengelypont n adatpont esetén:

$$m = \frac{n\sum x_k y_k - \sum x_k \sum y_k}{n\sum {x_k}^2 - (\sum x_k)^2}$$

$$\mathrm{b} = \frac{\sum \mathrm{y_k} - \mathrm{m}\sum \mathrm{x_k}}{\mathrm{n}}$$

A fejlesztés során kritikus szempont volt a hibatűrés. Ha a nevező értéke a meredekség képletében nulla (ami akkor fordul elő, amikor az összes X érték megegyezik; ebbe beletartozik a szélsőséges eset is, amikor n = 1), a PHP futtatókörnyezet végzetes hibával leállna. Ezt Zsigmond egy úgy kezelte le, hogy ilyenkor a meredekséget nullára állítja, az Y-metszetet pedig az egyetlen ismert Y értékkel teszi egyenlővé, tehát az adatokat egy konstans függvénnyel modellezi.

8.3. Predikció Integrálszámítással (Az accumulate metódus)

Amíg az evaluate() metódus csupán az egyenes egy adott jövőbeli pontját képes megmondani, a projekt megkövetelte, hogy a teljes várható *összkibocsátást* (tehát a görbe alatti területet) is megkapjuk.

Zsigmond ezt a problémát határozott integrálszámítással oldotta meg az accumulate() metódusban. Az összkibocsátás képlete adott időszakban a következő, ha a regressziós egyenest már kiszámoltuk:

$$\int_{t_0}^{t_1} (mx + b)dt = \frac{m}{2}(t_1^2 - t_0^2) \ + b(t_1 - t_0)$$

Ez a matematikai precizitás teszi lehetővé, hogy az Ecodrive rendkívül pontos becslést adjon a felhasználó év végi várható CO□-kibocsátásáról. Ezt az értéket Beni a frontendben vizuálisan, grafikonokon jeleníti meg, egyértelmű, vizsgarendszeri szinten is kiemelkedő visszajelzést adva az autósoknak arról, hogy adott időszakban (hónap végéig, év végéig) körülbelül mennyi szén-dioxidot fognak kibocsátani jelenlegi vezetési szokásaik megtartása mellett.

9. Modul: Összegzés és a Csapatmunka Értékelése

Az Ecodrive vizsgaremek elkészítése során a fő célunk egy olyan technológiailag érett, a modern szoftverfejlesztési elveket szem előtt tartó webalkalmazás létrehozása volt, amely nemcsak a vizsgakövetelményeknek felel meg, hanem a gyakorlatban is megállja a helyét. Visszatekintve az elmúlt hónapok munkájára, elmondhatjuk, hogy a kitűzött funkcionális és biztonsági célokat maradéktalanul teljesítettük.

A közös munka során természetesen szembesültünk technikai kihívásokkal. Ilyen volt például a kezdeti lokális környezetek inkompatibilitása, ami végül a Docker bevezetéséhez vezetett, vagy a komplex matematikai algoritmusok és SQL lekérdezések hibátlan integrálása. Ezeket az akadályokat azonban közös ötleteléssel és hatékony problémamegoldással sikeresen leküzdöttük, ami rengeteget tanított nekünk a valós szoftverfejlesztési folyamatokról.

Összességében az Ecodrive számunkra sokkal több lett egy kötelező vizsgafeladatnál. Egy olyan komplex projektet hoztunk létre, amelyen keresztül megtapasztaltuk a csapatmunka, a verziókezelés és a minőségbiztosítás fontosságát. Szakmailag mindannyian sokat fejlődtünk, és rendkívül büszkék vagyunk az elkészült szoftverre.

FORRÁSJEGYZÉK / FÁJLLISTA

1. Infrastruktúra és Konfiguráció

- **compose.dev.yml:** A Docker fejlesztői környezet konfigurációs fájlja (app-server, MySQL, phpMyAdmin szolgáltatások definiálása).
- **Dockerfile:** Az alkalmazásszerver (PHP 8.3 + Apache) egyedi felépítését és a jogosultságokat tartalmazó lemezkép-definíció.
- **.env:** A környezeti változókat és szenzitív adatokat (pl. adatbázis jelszó, hoszt) tároló konfigurációs fájl.
- **./app/config.php:** A központi konfigurációs osztály, amely a .env fájl beolvasásáért és az adatbázis-kapcsolat inicializálásáért felel.

2. Adatbázis

- **database.sql:** A MySQL relációs adatbázis sémája. Tartalmazza a táblák (users, sessions, vehicles, routes) létrehozását, az idegen kulcsokat és az alapértelmezett adminisztrátor (seed) adatait.

3. Adatmodellek (Models)

- **./app/models/User.php:** A felhasználói adatokat, jogosultságokat és az Argon2i jelszó-titkosítást (password_hash) kezelő osztály.
- **./app/models/Session.php:** A hagyományos fájlalapú munkamenetek helyett az adatbázis-vezérelt, biztonságos (HttpOnly) session tokenek kezeléséért felelős modell.
- **./app/models/Vehicle.php:** A gépjárművek adatait reprezentáló entitás, amely tartalmazza az európai átlagos CO2-kibocsátási normát (108.2 g/km) is.
- **./app/models/Route.php:** Az utazási napló bejegyzéseit és a járművekhez tartozó kibocsátási adatok (távolság * emissziós ráta) kiszámítását kezelő osztály.

4. Végpontok és Üzleti Logika (Endpoints)

- **./app/endpoints/Authentication.php:** A regisztrációs és bejelentkezési kéréseket feldolgozó (POST kérések) biztonsági végpont.
- **./app/endpoints/Vehicles.php:** A járművek létrehozásáért, frissítéséért és törléséért felelős végpont, amely tartalmazza az IDOR-védelmet és a komplex reguláris kifejezéseket.
- **./app/endpoints/Admin.php:** Az adminisztrációs felület műveleteit és a middleware-szerű jogosultság-ellenőrzést (isAdminPermissible) ellátó fájl.

5. Segédosztályok és Algoritmusok (Helpers)

- **./app/helpers/LeastSquaresExtrapolator.php:** A lineáris regressziót és az integrálszámítást (Legkisebb Négyzetek Módszere) megvalósító, tisztán natív PHP-ban írt statisztikai osztály.
- **./app/helpers/UserDataValidator.php:** A bemeneti adatok (felhasználónév, e-mail cím, jelszavak) szigorú formai követelményeit ellenőrző validációs segédosztály.